Day 05: June 16, 2026

Objectives for Today

- Establish communication between PX4 Autopilot in WSL2 and QGroundControl (QGC) on the Windows Host.
- Fix toolchain conflicts between Gazebo Classic and Gazebo Harmonic.
- Perform initial flight tests via the PX4 shell and map out the Multi-UAV Swarm Stigmergy algorithm.
- Set up Python-based offboard control using MAVSDK.

Tasks & Implementations

1. Networking & GCS Troubleshooting (PX4 ↔ QGroundControl)

- **The Problem:** QGroundControl showed a "Disconnected" state despite PX4 running smoothly in WSL2. MAVLink status showed rx: 0.0 B/s, indicating a broken, one-way connection.
- **Root Cause Analysis:** WSL2 was operating in its default Network Address Translation (NAT) mode, meaning inbound UDP packets from Windows were dropped before hitting WSL. Attempting to bridge it with netsh portproxy failed because it exclusively supports TCP, not UDP (which MAVLink utilizes).
- **The Fix:** 1. Configured WSL to use mirrored networking by creating a .wslconfig file in the Windows user profile:

Ini, TOML

[wsl2]

networkingMode=mirrored

2. Restarted WSL (wsl --shutdown).
3. Changed the Windows Wi-Fi network profile from **Public** to **Private** to lift local firewall restrictions.

- **Outcome:** MAVLink bidirectional handshake succeeded; GCS connected perfectly and arming checks cleared.

2. Gazebo Toolchain & Model Recovery

- **Issue A (Build Failure):** PX4 compilation failed with Gazebo gz sim not found. Diagnosed a conflict where a legacy *Gazebo Classic 11* install (exposing gz) was shadowing *Gazebo Harmonic 8.10.0*. Resolved by removing the legacy exposure path and explicitly symlinking/exposing gz from the Pixi-managed gz-harmonic environment.
- **Issue B (Model Corruption):** The drone failed to spawn with a Failed to find top level <model> error. Used git status --porcelain to discover that Gazebo's GUI had accidentally overwritten the tracked x500/model.sdf file.
- **The Fix:** Restored the pristine source file using git checkout -- daily-logs/ paths and established a team rule: *Never save directly over PX4 source model/world files; always use "Save World As" to a custom directory.*

3. Simulation Flight Testing & Parameter Tuning

- **Manual Control:** Successfully executed takeoff and landing sequences within the Gazebo Baylands world using the PX4 shell interpreter:

Bash

commander arm

commander takeoff

commander land

- Verified active flight states using commander status and tracked coordinate data over listener vehicle_local_position.
- **Edge Cases Identified:** Documented that commander arm yields no response if the system is already armed, and disarm is safety-blocked mid-air if the system registers a "not landed" state.
- **Parameter Mapping:** Attempted adjusting takeoff behavior using MPC_TAKEOFF_ALT. Discovered this parameter is deprecated or absent in this PX4 release; shifted search parameters to locate its current equivalent (MIS_TAKEOFF_ALT).

4. MAVSDK Python Offboard Control Setup

- Installed mavsdk via pip on Python 3.14 and verified compilation.
- Drafted an initial Python offboard control script to handle automated flight trajectories.
- **Network Binding Adjustments:** Resolved a udp:// syntax deprecation warning and a port conflict by shifting the listener protocol to udpin://0.0.0.0:14540.
- Altered North-East-Down (NED) frame coordinates (Z values) to dynamically test higher-altitude holds.

5. Swarm Architecture: SMA + Stigmergy Algorithm

- Evaluated our comprehensive **Jamming-Aware Hybrid Metaheuristic with Pheromone-Based Stigmergy** swarm protocol.
- Formulated a systematic **7-Step Implementation Pipeline** starting with a lightweight, standalone 2D Python script simulation to validate pheromone mathematics before introducing heavy physics/hardware-in-the-loop dependencies (MAVSDK/PX4/Gazebo).

🚧 **Challenges & Blockers**

- **In-Sim Mishap:** The simulated drone collided with a tree asset in Gazebo and got physically stuck. Handled this via a manual pose-reset and a clean simulation environment restart.
- **Critical Blocker (Unresolved):** The entire WSL2 filesystem (or specifically the robotics_sim directory) unexpectedly mounted as **Read-Only** following an unclean shutdown/frozen Gazebo thread. This is currently blocking text editors (nano, vim) from modifying files, temporarily halting MAVSDK script deployment.

Key Learnings & Takeaways

- **WSL2 Architecture:** Traditional NAT configurations in virtualized layers severely break UDP-heavy robotics frameworks. Mirrored networking mode is highly critical for multi-agent ROS/PX4 architectures.
- **Git Integrity:** Keeping tracking repositories totally clean allows for rapid disaster recovery when simulation tools stealthily modify source geometry or SDF config files.

Plan for Tomorrow

- Resolve the WSL read-only filesystem bug (check dmesg, check drive corruption, or remount using sudo mount -o remount,rw /).
- Complete execution of the initial MAVSDK Python script once file access is restored.
- Begin constructing the mathematical engine for the standalone 2D Stigmergy prototype